

Practical 1: Python Libraries for AI Development

Objective

To understand and implement basic usage of essential Python libraries commonly used in Artificial Intelligence, Machine Learning, and Web Development.

Theory

Python has become the most used language for AI development due to its rich ecosystem of libraries. Understanding these libraries is crucial for any AI practitioner:

Core Libraries Overview:

1. LangChain & LangGraph

- **LangChain:** Framework for developing applications powered by language models
- **LangGraph:** Extension for building stateful, multi-actor applications with LLMs
- **Use Cases:** Chatbots, document analysis, conversational AI

2. Data Science Libraries

- **Pandas:** Data manipulation and analysis library
- **NumPy:** Fundamental package for scientific computing
- **Matplotlib:** Comprehensive library for creating static, animated, and interactive visualizations

3. Machine Learning Libraries

- **scikit-learn:** Simple and efficient tools for predictive data analysis
- **TensorFlow:** End-to-end open source platform for machine learning

4. Computer Vision

- **OpenCV:** Library of programming functions mainly aimed at real-time computer vision

5. Web Development Frameworks

- **Streamlit:** Framework for creating data apps
- **FastAPI:** Modern, fast web framework for building APIs
- **Django:** High-level Python web framework

Python Library Implementations

This notebook demonstrates basic usage of the following libraries:

- LangChain
- LangGraph
- Matplotlib
- Pandas
- NumPy
- scikit-learn
- TensorFlow
- OpenCV
- Streamlit
- FastAPI
- Django

Each code cell below shows a minimal example for each library.

```
In [18]: from langchain_google_genai import ChatGoogleGenerativeAI
from dotenv import load_dotenv
import os
load_dotenv()

# Initialize the Gemini model
gemini_api = os.getenv("GEMINI-API")

llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash-lite",
    google_api_key=gemini_api,
    temperature=0.2
)

# Simple call
input_question = "explain the concept of reinforcement learning in AI in 1 para or less"
response = llm.invoke(input_question)
print(response.content)
```

Reinforcement learning is a type of machine learning where an agent learns to make decisions by performing actions in an environment to maximize a cumulative reward. It learns through trial and error, receiving positive rewards for desirable actions and negative rewards (or penalties) for undesirable ones, gradually improving its strategy over time.

```
In [5]: from langgraph.graph import StateGraph
from typing import Dict, TypedDict

class GraphState(TypedDict):
    input: str
    result: str

# 2. Initialize the StateGraph with the state definition.
graph = StateGraph(GraphState)

# 3. Define your graph's nodes (as functions).
def start_node(state: GraphState):
    print("---STARTING---")
    return {"result": "The process has started."}

def end_node(state: GraphState):
```

```

print("---ENDING---")
print(f"Final Input: {state['input']}")
print(f"Final Result: {state['result']}")
return {}

# 4. Add the nodes to the graph.
graph.add_node("start", start_node)
graph.add_node("end", end_node)

# 5. Define the edges that connect the nodes.
graph.add_edge("start", "end")

# 6. Set the entry point for the graph.
graph.set_entry_point("start")
graph.set_finish_point("end")

# 7. Compile the graph into a runnable object.
runnable_graph = graph.compile()

# 8. Run the graph!
final_state = runnable_graph.invoke({"input": "Hello, LangGraph!"})

---STARTING---
---ENDING---
Final Input: Hello, LangGraph!
Final Result: The process has started.

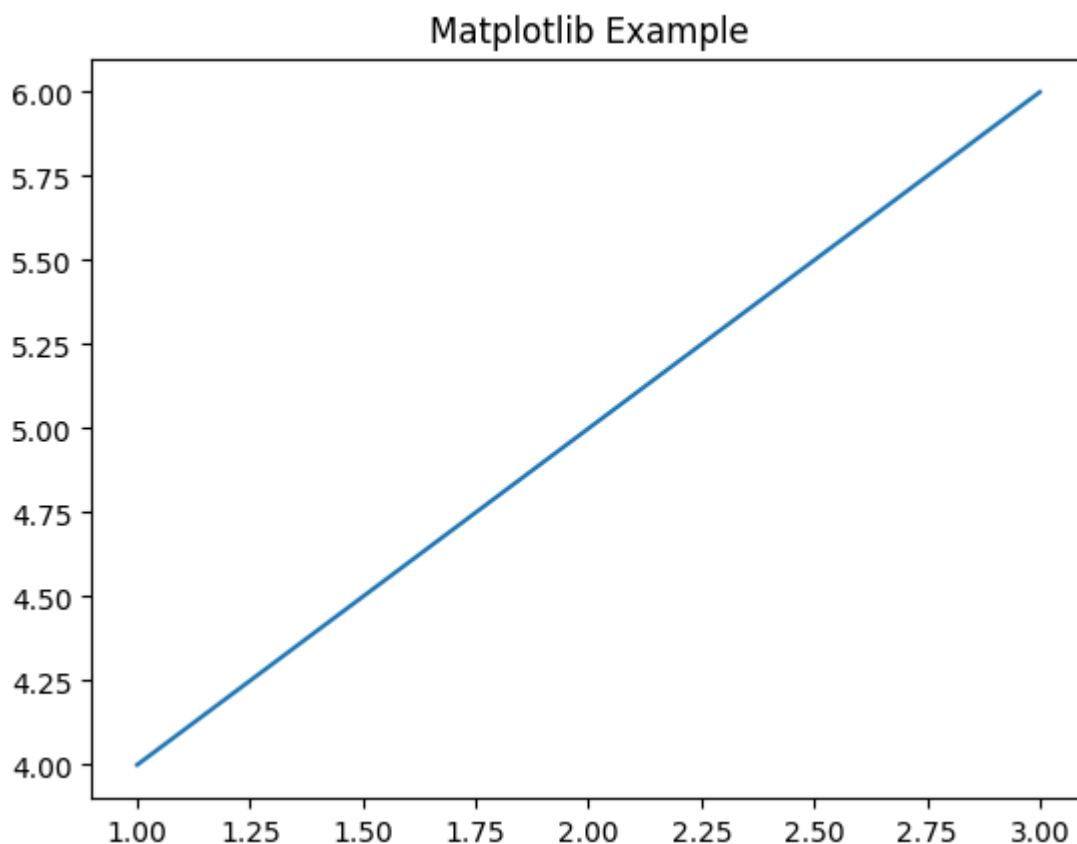
```

In [6]:

```

# Matplotlib Example
import matplotlib.pyplot as plt
plt.plot([1,2,3],[4,5,6])
plt.title("Matplotlib Example")
plt.show()

```



In [7]:

```

# Pandas Example
import pandas as pd
df = pd.DataFrame({'A': [1,2], 'B': [3,4]})
print(df)

```

```
A B 0
1 3 1 2
4
```

```
In [8]: # NumPy Example
import numpy as np
arr = np.array([1,2,3])
print(arr * 2)
```

```
[2 4 6]
```

```
In [9]: # scikit-learn Example
from sklearn.linear_model import LinearRegression
model = LinearRegression()

X = [[0],[1],[2]] y = [0,1,2]
model.fit(X, y)
print(model.predict([[3]]))
```

```
[3.]
```

```
In [15]: from warnings import filterwarnings
filterwarnings("ignore")

# TensorFlow Example
import tensorflow as tf a
= tf.constant(2)
b = tf.constant(3)
print(tf.add(a, b))
```

```
tf.Tensor(5, shape=(), dtype=int32)
```

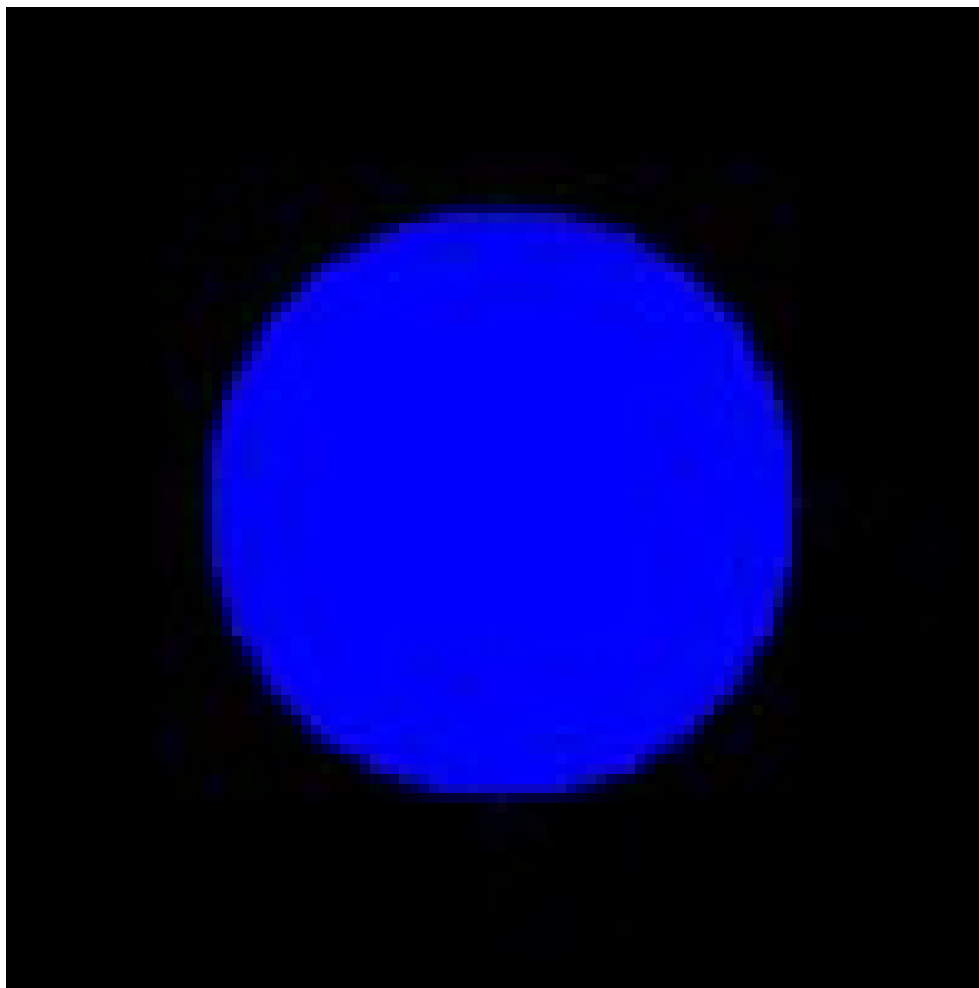
```
In [16]: # Streamlit Example (run with: streamlit run <filename>)
import streamlit as st
st.write("Hello, Streamlit!")
```

```
2025-10-05 09:13:29.655 Thread 'MainThread': missing ScriptRunContext! This warning ca n be
ignored when running in bare mode.
2025-10-05 09:13:29.659 Thread 'MainThread': missing ScriptRunContext! This warning ca n be
ignored when running in bare mode.
2025-10-05 09:13:29.661 Thread 'MainThread': missing ScriptRunContext! This warning ca n be
ignored when running in bare mode.
2025-10-05 09:13:29.659 Thread 'MainThread': missing ScriptRunContext! This warning ca n be
ignored when running in bare mode.
2025-10-05 09:13:29.661 Thread 'MainThread': missing ScriptRunContext! This warning ca n be
ignored when running in bare mode.
```

```
In [13]: # FastAPI Example (run with: uvicorn <filename>:app) from
fastapi import FastAPI
app = FastAPI()
@app.get("/")
def read_root():
    return {"Hello": "FastAPI"}
```

```
In [14]: # Django Example (run with: python manage.py runserver) from
django.http import HttpResponse
def hello(request):
    return HttpResponse("Hello, Django!")
```

```
In [11]: # OpenCV Example
import cv2
import numpy as np
img = np.zeros((100,100,3), np.uint8)
cv2.circle(img, (50,50), 30, (255,0,0), -1)
cv2.imwrite('circle.png', img)
img = cv2.imread('circle.png')
cv2.imshow('Circle', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Circle.png